AUTONOMOUS MULTI-AGENT SYSTEMS • ENTERPRISE COMPLIANCE AI

RegRadar: Multi-Agent Regulatory Change Radar

Automated Regulatory Ingestion, Control Mapping, Adversarial Gap Validation & Remediation Planning

Domain: Fintech / Payment Aggregator (RBI • MeitY) **Engine:** LangGraph StateGraph • Groq (120B LLM) **Stack:** Python 3.12, FastAPI, React 19, Vercel Multi-Service

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

Digital financial entities operate under dense, rapidly shifting regulatory mandates from authorities such as the Reserve Bank of India (RBI), SEBI, and MeitY. Compliance teams face three critical operational bottlenecks:

1. **Analysis Latency:** Complex regulatory circulars require hundreds of human hours to manually parse into atomic obligations.
2. **Quantitative Shortfall Blindspots:** Qualitative audits often confirm high-level alignment (e.g. "an incident response procedure exists") while overlooking strict quantitative parameter gaps (e.g. regulation requires a 6-hour reporting window while internal policy enforces 24 hours).
3. **LLM Hallucination Risk:** Single-prompt LLM agents suffer from ungrounded claims, invented citations, and premature acceptance of partial controls.

RegRadar solves this via a cooperative, adversarial multi-agent architecture built on **LangGraph**. Six specialized autonomous agents retrieve circulars, extract atomic obligations, compute coverage matrices against internal controls, synthesize gaps, rigorously stress-test each gap across **nine authoritative validation checks** in an adaptive feedback loop, and produce prioritised remediation plans. RegRadar operates in both **Live LLM Mode** (powered by Groq 120B reasoning) and **Deterministic Demo Mode** (repeatable synthetic evaluation).

2. ARCHITECTURE & THE SIX SPECIALIZED AGENTS

```
START → [Regulatory Scout] → [Obligation Extractor] → [Control Mapper] → [Gap Synthesiser]
                                                                              ↑            ↓
                                                                [Revise Loop]   [Novelty Validator]
                                                                                  ↓ (conditional route)
                                                                [Escalation (Max Rev)] / [Finalise]
                                                                                              ↓
                                                                                [Output Agent] → END
```

#	Agent Name	Role & Core Methodology	Output Artifact
1	**Regulatory Scout**	Ingests circulars (synthetic dataset, user pasted text, or live RSS feeds) and scores relevance.	`documents`
2	**Obligation Extractor**	Decomposes clauses into atomic obligations, detects deontic modals (*shall/must* → mandatory), regex-extracts quantitative parameters, enforces strict verbatim grounding.	`obligations`
3	**Control Mapper**	Computes hybrid similarity ($0.65 \times \text{TF-IDF Cosine} + 0.35 \times \text{Keyword Overlap}$) and evaluates parameter shortfalls against internal controls.	`coverage` matrix
4	**Gap Synthesiser**	Proposes actionable gaps. In feedback loops, programmatically interprets validator rejection codes to revise severity, quote verbatim evidence, or withdraw.	`gaps` proposals
5	**Novelty Validator**	Adversarial quality gate enforcing 9 deterministic rules (evidence grounding, novelty, duplicate check, severity calibration). Issues structured feedback.	`validation_decisions` & `pending_feedback`

#	Agent Name	Role & Core Methodology	Output Artifact
6	**Output Agent**	Calculates multi-factor risk scores ($R = S \times U \times E \times C$), determines P1/P2/P3 SLAs and departmental owners, and writes executive Markdown reports.	`remediation_plan` & `report`

3. KEY INNOVATIONS & ENGINEERING HIGHLIGHTS

- **Deterministic Business Rules Override LLMs:** LLMs provide generative and syntactic extraction capabilities, but the Novelty Validator's deterministic code rules maintain absolute veto authority to eliminate hallucinations.
- **Structured Machine-Readable Feedback Contracts:** Rejections return structured issues (`code`, `field`, `expected`, `message`). The Gap Synthesiser parses these codes to converge automatically without manual prompts.
- **Bounded Revision with Graceful Escalation:** Cycles are bounded by `max_revisions` (default: 2). Unresolved disputes escalate directly to human review with complete reasoning audit logs.
- **Production Live LLM Integration:** Integrated with Groq's high-speed inference engine (`openai/gpt-oss-120b`) with automated Cloudflare client signatures and resilient JSON self-repair.
- **Modern Light Theme UI & Cloud Deployment:** Features a crisp, high-contrast light theme with real-time dark/light switching and multi-service deployment on Vercel.

4. VERIFICATION & TEST COVERAGE

The system is backed by **64 automated pytest tests** covering unit logic, agent routing, feedback convergence, and API contracts. End-to-end verified via `scripts/verify.ps1` with 100% pass rate.